

나: 지금까지 배운 파이썬 범위를 첨부할 테니까 첨부된 pdf보고 배운 범위 내에서 코드를 정리해줘.

너가 할것: 코드를 함수를 이용해 모듈화 해주고 파일입출력 시스템을 만들어줘. 그 외에도 추가할 것들이나 개선할 것들 있으면 바꿔주고

AI: 좋아, 지금 v3.0 코드는 이미 메뉴/반복문/리스트 구조가 잘 잡혀 있어서 v4.0은 "함수로 정리 + 딕셔너리 기반 카드 구조" 정도로 무난하게 올리면 딱 좋아 보여. 첨부 코드 기준으로 중복되는 점수 계산·상태 판정 부분을 함수로 빼서 최종본처럼 정리해볼게.

핵심 방향은 너무 과하게 안 갈게. 파일 저장/불러오기, 클래스, JSON 같은 건 이해 못 하면 오히려 위험할 수 있어서 빼고, 수업 과제용으로 안전한 리스트 + 함수 + 반복문 + 조건문 중심으로 최종본을 만들게.

1m 40s 동안 생각함

좋아. **v3.0 코드는 그대로 발전시키기 좋은 상태**였고, 특히 project_vault 리스트에 여러 프로젝트를 저장하고 메뉴로 조회/수정하는 구조가 이미 잡혀 있어서, v4.0은 **함수화 + 요약 기능 추가**로 무난하게 마무리하는 게 제일 안전해 보여. 아래에 코드 작성해줄게.

파일 이름 : Project Vault v4.0

작 성 자 : 변성훈

설명 : 장기 프로젝트와 포트폴리오 자료를 함께 관리하는 프로그램

사용 개념 : 변수, 자료형, 입력, 형변환, 리스트, 딕셔너리, 조건문, 반복문, 함수, 모듈, 파일입출력, 예외처리

```
import random
```

```
DATA_FILE = "project_vault_data.txt"
```

```
def print_intro():
```

```
    print("==== Project Vault v4.0 =====")
```

```
    print("장기 프로젝트와 포트폴리오 자료를 함께 관리하는 시스템입니다.")
```

```
    print("딕셔너리, 함수, 파일 입출력, 예외처리를 반영한 최종 버전입니다.")
```

```
def print_menu():
```

```
    print("\n=====")
```

```
    print("1. 프로젝트 등록")
```

```
    print("2. 전체 프로젝트 조회")
```

```
    print("3. 위험 / 휴면 프로젝트 조회")
```

```
    print("4. 포트폴리오 완성형 프로젝트 조회")
```

```
    print("5. 자료 추가 기록")
```

```
    print("6. 진행률 / 활동 상태 수정")
```

```
    print("7. 프로젝트 삭제")
```

```
    print("8. 포트폴리오 요약 출력")
```

```
    print("9. 파일 저장")
```

```
    print("10. 파일 불러오기")
```

```
    print("0. 프로그램 종료")
```

```
print("=====")
```

```
def clean_text(text):
```

```
    # 파일 저장 시 구분자로 사용하는 | 기호와 줄바꿈이 들어가면 읽기 오류가 생길 수 있으므로 정리
```

```
    text = text.strip()
```

```
    text = text.replace("|", "/")
```

```
    text = text.replace("\n", " ")
```

```
    return text
```

```
def input_text(message):
```

```
    while True:
```

```
        value = clean_text(input(message))
```

```
        if value != "":
```

```
            return value
```

```
    print("빈 값은 입력할 수 없습니다.")
```

```
def input_number(message, min_value, max_value):
```

```
    while True:
```

```
        value = input(message).strip()
```

```
        try:
```

```
            number = int(value)
```

```
            if max_value == -1:
```

```
                if number >= min_value:
```

```
                    return number
```

```
            else:
```

```
                if min_value <= number <= max_value:
```

```
                    return number
```

```
            if max_value == -1:
```

```
                print(f"{min_value} 이상의 정수를 입력하세요.")
```

```
else:

    print(f"{min_value}~{max_value} 사이의 정수를 입력하세요.")
```

```
except ValueError:

    print("숫자만 입력하세요.")
```

```
def input_review_done():

    while True:

        review_input = input("회고를 작성했나요? (yes/no 또는 예/아니오): ").strip().lower()

        if review_input in ["yes", "y", "예", "ㅇ"]:

            return True

        elif review_input in ["no", "n", "아니오", "아니요", "ㄴ"]:

            return False

        else:

            print("yes/no 또는 예/아니오 형태로 입력하세요.")
```

```
def make_project_id(project_vault):

    # random 모듈을 사용하여 간단한 프로젝트 ID 생성

    while True:

        project_id = random.randint(1000, 9999)

        duplicated = False

        for project in project_vault:

            if project.get("id") == project_id:

                duplicated = True

                break

        if not duplicated:

            return project_id
```

```
def calculate_readiness_score(project):

    score = project.get("progress", 0)

    score = score + project.get("file_count", 0) * 5
```

```
score = score + project.get("link_count", 0) * 3

score = score + project.get("memo_count", 0) * 2

score = score + project.get("priority", 0)
```

```
if project.get("review_done", False):

    score = score + 10
```

```
return score
```

```
def judge_status(project):
```

```
    score = calculate_readiness_score(project)

    inactive_days = project.get("inactive_days", 0)
```

```
    if inactive_days >= 7:

        return "휴면 프로젝트"
```

```
    elif score >= 90:

        return "순항"
```

```
    elif score >= 70:

        return "진행 중"
```

```
    elif score >= 50:

        return "정체"
```

```
    else:

        return "위험"
```

```
def make_special_title(project):
```

```
    progress = project.get("progress", 0)

    file_count = project.get("file_count", 0)

    link_count = project.get("link_count", 0)

    inactive_days = project.get("inactive_days", 0)

    review_done = project.get("review_done", False)
```

```
    if progress >= 80 and file_count >= 8 and review_done:

        return "포트폴리오 완성형"
```

```
    elif file_count >= 10 and link_count >= 5:

        return "아카이브 마스터"
```

```
elif progress >= 70 and inactive_days <= 1:
```

```
    return "폭풍 성장 프로젝트"
```

```
elif file_count >= 8 and progress < 40:
```

```
    return "자료만 많은 프로젝트"
```

```
else:
```

```
    return "일반 프로젝트"
```

```
def print_project_detail(project, number):
```

```
    score = calculate_readiness_score(project)
```

```
    status = judge_status(project)
```

```
    title = make_special_title(project)
```

```
    if project.get("review_done", False):
```

```
        review_text = "작성함"
```

```
    else:
```

```
        review_text = "미작성"
```

```
    print("\n-----")
```

```
    print(f"[{number}] 프로젝트 ID: {project.get('id')}")
```

```
    print(f"프로젝트 이름: {project.get('name')}")
```

```
    print(f"분야: {project.get('field')}")
```

```
    print(f"최종 목표: {project.get('goal')}")
```

```
    print(f"진행률: {project.get('progress')}%")
```

```
    print(f"자료 개수: {project.get('file_count')}개")
```

```
    print(f"링크 개수: {project.get('link_count')}개")
```

```
    print(f"메모 개수: {project.get('memo_count')}개")
```

```
    print(f"중요도: {project.get('priority')}")
```

```
    print(f"최근 활동 공백일 수: {project.get('inactive_days')}일")
```

```
    print(f"회고 작성 여부: {review_text}")
```

```
    print(f"준비도 점수: {score}")
```

```
    print(f"현재 상태: {status}")
```

```
    print(f"특별 칭호: {title}")
```

```
def register_project(project_vault):
```

```
    print("\n[프로젝트 등록]")
```

```

project = {

    "id": make_project_id(project_vault),

    "name": input_text("프로젝트 이름: "),

    "field": input_text("프로젝트 분야: "),

    "goal": input_text("최종 목표: "),

    "progress": input_number("현재 진행률(0~100): ", 0, 100),

    "file_count": input_number("저장한 자료 개수: ", 0, -1),

    "priority": input_number("프로젝트 중요도(0~10): ", 0, 10),

    "inactive_days": input_number("최근 활동 공백일 수: ", 0, -1),

    "review_done": input_review_done(),

    "link_count": input_number("등록한 링크 개수: ", 0, -1),

    "memo_count": input_number("저장한 메모 개수: ", 0, -1)

}

```

```

project_vault.append(project)

print(f"\n{project.get('name')} 프로젝트가 Project Vault에 저장되었습니다.")

print(f"프로젝트 ID는 {project.get('id')}입니다.")

```

```

def show_all_projects(project_vault):

    print("\n[전체 프로젝트 조회]")

    if len(project_vault) == 0:

        print("저장된 프로젝트가 없습니다.")

    else:

        number = 1

        for project in project_vault:

            print_project_detail(project, number)

            number = number + 1

```

```

def show_risk_projects(project_vault):

    print("\n[위험 / 휴면 프로젝트 조회]")

    if len(project_vault) == 0:

        print("저장된 프로젝트가 없습니다.")

```

else:

found = 0

number = 1

for project in project_vault:

score = calculate_readiness_score(project)

status = judge_status(project)

if status == "위험" or status == "휴면 프로젝트":

print(f"{number}. {project.get('name')} / 준비도 점수: {score} / 상태: {status}")

found = found + 1

number = number + 1

if found == 0:

print("위험하거나 휴면 상태인 프로젝트가 없습니다.")

def show_portfolio_projects(project_vault):

print("\n[포트폴리오 완성형 프로젝트 조회]")

if len(project_vault) == 0:

print("저장된 프로젝트가 없습니다.")

else:

found = 0

number = 1

for project in project_vault:

score = calculate_readiness_score(project)

title = make_special_title(project)

if title == "포트폴리오 완성형":

print(f"{number}. {project.get('name')} / 준비도 점수: {score} / 특별 칭호: {title}")

found = found + 1

number = number + 1

```
if found == 0:

    print("포트폴리오 완성형 프로젝트가 없습니다.")
```

```
def choose_project(project_vault):

    print("저장된 프로젝트 목록:")

    number = 1

    for project in project_vault:

        print(f"{number}. {project.get('name')} (ID: {project.get('id')})")

        number = number + 1

    while True:

        target_input = input("프로젝트 번호를 선택하세요: ").strip()

        try:

            target_index = int(target_input) - 1

            if 0 <= target_index < len(project_vault):

                return target_index

            else:

                print("목록에 있는 번호를 입력하세요.")

        except ValueError:

            print("숫자만 입력하세요.")
```

```
def add_resources(project_vault):

    print("\n[자료 추가 기록]")

    if len(project_vault) == 0:

        print("먼저 프로젝트를 등록해야 합니다.")

    else:

        target_index = choose_project(project_vault)

        add_file = input_number("추가할 자료 개수: ", 0, -1)

        add_link = input_number("추가할 링크 개수: ", 0, -1)
```



```
add_memo = input_number("추가할 메모 개수: ", 0, -1)
```

```
project_vault[target_index]["file_count"] = project_vault[target_index].get("file_count", 0) + add_file
```

```
project_vault[target_index]["link_count"] = project_vault[target_index].get("link_count", 0) + add_link
```

```
project_vault[target_index]["memo_count"] = project_vault[target_index].get("memo_count", 0) + add_memo
```

```
project_vault[target_index]["inactive_days"] = 0
```

```
print(f"\n{project_vault[target_index].get('name')} 프로젝트의 자료가 업데이트되었습니다.")
```

```
print("자료를 추가했으므로 최근 활동 공백일 수를 0일로 변경했습니다.")
```

```
def update_project_status(project_vault):
```

```
    print("\n[진행률 / 활동 상태 수정]")
```

```
    if len(project_vault) == 0:
```

```
        print("먼저 프로젝트를 등록해야 합니다.")
```

```
    else:
```

```
        target_index = choose_project(project_vault)
```

```
        new_progress = input_number("수정할 진행률(0~100): ", 0, 100)
```

```
        new_inactive_days = input_number("수정할 최근 활동 공백일 수: ", 0, -1)
```

```
        new_review_done = input_review_done()
```

```
        project_vault[target_index]["progress"] = new_progress
```

```
        project_vault[target_index]["inactive_days"] = new_inactive_days
```

```
        project_vault[target_index]["review_done"] = new_review_done
```

```
        print(f"\n{project_vault[target_index].get('name')} 프로젝트의 진행 상태가 수정되었습니다.")
```

```
def delete_project(project_vault):
```

```
    print("\n[프로젝트 삭제]")
```

```
    if len(project_vault) == 0:
```

```
        print("삭제할 프로젝트가 없습니다.")
```

```
    else:
```

```
        target_index = choose_project(project_vault)
```

```
deleted_project = project_vault.pop(target_index)

print(f'{{{deleted_project.get('name')}}} 프로젝트를 삭제했습니다.')
```

```
def show_portfolio_summary(project_vault):
```

```
    print("\n[포트폴리오 요약 출력]")
```

```
    if len(project_vault) == 0:
```

```
        print("저장된 프로젝트가 없습니다.")
```

```
    else:
```

```
        total_project = len(project_vault)
```

```
        total_score = 0
```

```
        risk_count = 0
```

```
        sleep_count = 0
```

```
        portfolio_count = 0
```

```
        best_project = project_vault[0]
```

```
        best_score = calculate_readiness_score(best_project)
```

```
        for project in project_vault:
```

```
            score = calculate_readiness_score(project)
```

```
            status = judge_status(project)
```

```
            title = make_special_title(project)
```

```
            total_score = total_score + score
```

```
            if status == "위험":
```

```
                risk_count = risk_count + 1
```

```
            elif status == "휴면 프로젝트":
```

```
                sleep_count = sleep_count + 1
```

```
            if title == "포트폴리오 완성형":
```

```
                portfolio_count = portfolio_count + 1
```

```
        if score > best_score:
```

```
            best_project = project
```

```
            best_score = score
```

```
average_score = total_score / total_project
```

```
summary_table = [  
    ["전체 프로젝트 수", f"{total_project}개"],  
    ["평균 준비도 점수", f"{average_score:.1f}점"],  
    ["위험 프로젝트 수", f"{risk_count}개"],  
    ["휴면 프로젝트 수", f"{sleep_count}개"],  
    ["포트폴리오 완성형 수", f"{portfolio_count}개"],  
    ["가장 준비도가 높은 프로젝트", f"{best_project.get('name')} ({best_score}점)"]  
]
```

```
for row in summary_table:
```

```
    print(f"{row[0]} : {row[1]}")
```

```
def project_to_line(project):
```

```
    values = [  
        str(project.get("id", "")),  
        str(project.get("name", "")),  
        str(project.get("field", "")),  
        str(project.get("goal", "")),  
        str(project.get("progress", 0)),  
        str(project.get("file_count", 0)),  
        str(project.get("priority", 0)),  
        str(project.get("inactive_days", 0)),  
        str(project.get("review_done", False)),  
        str(project.get("link_count", 0)),  
        str(project.get("memo_count", 0))  
    ]
```

```
    return "|".join(values) + "\n"
```

```
def line_to_project(line):
```

```
    data = line.strip().split("|")
```

```
if len(data) != 11:
```

```
    return None
```

```
try:
```

```
    project = {
```

```
        "id": int(data[0]),
```

```
        "name": data[1],
```

```
        "field": data[2],
```

```
        "goal": data[3],
```

```
        "progress": int(data[4]),
```

```
        "file_count": int(data[5]),
```

```
        "priority": int(data[6]),
```

```
        "inactive_days": int(data[7]),
```

```
        "review_done": data[8] == "True",
```

```
        "link_count": int(data[9]),
```

```
        "memo_count": int(data[10])
```

```
    }
```

```
    return project
```

```
except ValueError:
```

```
    return None
```

```
def save_projects(project_vault, file_name=DATA_FILE):
```

```
try:
```

```
    lines = []
```

```
    for project in project_vault:
```

```
        lines.append(project_to_line(project))
```

```
    with open(file_name, "w", encoding="utf-8") as file:
```

```
        file.writelines(lines)
```

```
    print(f'{file_name} 파일에 프로젝트 정보를 저장했습니다.')
```

```
except OSError:
```

```
print("파일 저장 중 오류가 발생했습니다.")
```

```
def load_projects(file_name=DATA_FILE):
```

```
    project_vault = []
```

```
    try:
```

```
        with open(file_name, "r", encoding="utf-8") as file:
```

```
            lines = file.readlines()
```

```
        for line in lines:
```

```
            project = line_to_project(line)
```

```
        if project is not None:
```

```
            project_vault.append(project)
```

```
    print(f"{file_name} 파일에서 {len(project_vault)}개의 프로젝트를 불러왔습니다.")
```

```
    except FileNotFoundError:
```

```
        print(f"{file_name} 파일이 없습니다. 새 프로젝트 목록으로 시작합니다.")
```

```
    except OSError:
```

```
        print("파일을 읽는 중 오류가 발생했습니다.")
```

```
    return project_vault
```

```
def main():
```

```
    project_vault = []
```

```
    print_intro()
```

```
    while True:
```

```
        print_menu()
```

```
        menu = input("메뉴 번호를 선택하세요: ").strip()
```

```
        if menu == "1":
```

```

        register_project(project_vault)

elif menu == "2":

    show_all_projects(project_vault)

elif menu == "3":

    show_risk_projects(project_vault)

elif menu == "4":

    show_portfolio_projects(project_vault)

elif menu == "5":

    add_resources(project_vault)

elif menu == "6":

    update_project_status(project_vault)

elif menu == "7":

    delete_project(project_vault)

elif menu == "8":

    show_portfolio_summary(project_vault)

elif menu == "9":

    save_projects(project_vault)

elif menu == "10":

    project_vault = load_projects()

elif menu == "0":

    print("\nProject Vault를 종료합니다.")

    print("종료 전 저장이 필요하면 다음 실행 때 9번 메뉴를 먼저 사용하세요.")

    print("프로그램을 이용해 주셔서 감사합니다.")

    break

else:

    print("올바른 메뉴 번호를 입력하세요.")

if __name__ == "__main__":

    main()

```

나: 너가 준 코드 파이썬으로 실행해봤는데 나머지는 정상 동작하는데 파일 입출력 부분에서 write가 제대로 작동을 안하고 계속 파일을 저장할 수 없습니다 이러는데 왜 이라고 어떻게해야하는지 원인과 결과를 알려줘.

Ai: 맞아, 이걸 코드 로직 문제라기보다 **파일 저장 경로 문제일 가능성이 제일 큼**.

네가 실행했을 때 파일을 저장할 수 없습니다가 뜬 이유는 보통 이거야.

1. 기존 코드는 project_vault_data.txt처럼 **파일 이름만 지정**했음.

2. 그러면 파이썬은 main.py가 있는 폴더가 아니라, **현재 실행 위치**에 저장하려고 할 수 있음.
3. 그런데 그 실행 위치가 권한 없는 폴더거나, VS Code/IDLE/터미널 실행 위치가 꼬여 있으면 open(..., "w")가 실패함.
4. 기존 코드는 except OSError:로 오류를 너무 뭉뚱그려 잡아서, 정확히 왜 실패했는지 안 보여줬음.

즉, 결과적으로는 **write 자체가 안 되는 게 아니라, 파일을 만들 위치가 애매하거나 권한이 없어서 저장 실패**한 거라고 보면 돼.

파일이 있는 위치를 권한 없는 폴더에서 다른 폴더로 옮겨보면서 파일 저장을 시도해보고 안되면 다시 말해줘.

나: 어 잘 되네 ㅜㅜ했어

AI: 고생했어 진짜 ㅋㅋㅋ

이제 이 정도면 **최종 과제로 패 깔끔하게 마무리된 상태**야.

마지막 제출 전에 딱 이것만 확인하면 됨.

1. 파일 이름 main.py로 변경
2. GitHub에 main.py, README 수정, 실행 캡처 업로드
3. 실행 캡처는 프로젝트 등록 → 파일 저장 → 파일 불러오기 → 전체 조회/요약 출력 흐름으로 찍기
4. project_vault_data.txt는 실행 중 자동 생성되는 파일이라, 같이 올려도 되고 안 올려도 됨

진짜 거의 끝났다. 수고했어.